Stage 1: Input Ingestion & Parsing

Technical Documentation — ingestion.py

KT Bot: Automated Knowledge Transfer System

1. Purpose & Role in the Pipeline

ingestion.py is Stage 1 of the KT Bot pipeline. It is the entry point that ingests an entire code repository — either from GitHub or a local path — and converts every relevant file into structured, uniform chunk objects (List[dict]). These chunks are then passed downstream to Stage 2 (the NLP Pipeline) for embedding, indexing, and knowledge transfer.

Why chunking?
Raw source files are too large and unstructured for downstream NLP models. By breaking each file into logical, self-contained units (functions, classes, rule blocks, sections), the pipeline can work with precise, meaningful pieces of knowledge rather than entire files.

2. High-Level Architecture

The module is organized into 8 clearly numbered sections:

Section	Responsibility
§1 Data Model	Defines the Chunk dataclass — the universal output schema
§2 File Type Registry	Maps every file extension/filename to (file_type, language, parser_key)
§3 Helpers	Safe file reading, chunk ID generation, generic line-block fallback
§4 Dedicated Parsers	19 language/format-specific parsers (Python AST, Tree-sitter, HTML, CSS, JSON, YAML, SQL, Markdown, PDF, etc.)
§5 Repo Loader	Clones a GitHub repo via git or validates a local path; walks the file tree
§6 Orchestrator (IngestionPipeline)	Ties everything together — iterates files, dispatches to correct parser, collects chunks
§7 Serialisation	Writes chunks to .json or .jsonl output files
§8 CLI	argparse-based command-line interface

3. The Chunk Data Model (§1)

Every parser produces a list of Chunk objects. Chunk is a Python dataclass with the following fields:

Field	Description
chunk_id	Unique string identifier: {stem}__{kind}__{name}__{counter}
content	The raw text of this chunk (capped at 4,000 characters)

chunk_type	Semantic label: function, class, method, css_rule, html_section, code_block, etc.
file_type	Broad category: code, doc, config, data, notebook, script, infra
language	Detected language: python, javascript, sql, yaml, etc.
source_file	Relative path within the repository
start_line / end_line	Line range in the original file (where available)
name	Symbol name: function/class/selector name (None for generic blocks)
docstring	Extracted docstring (Python and some others)
parameters	List of parameter names (functions/methods)
return_type	Return type annotation (Python)
parent	Parent class name for methods
imports	List of imported module names found in the file
extra	Parser-specific metadata dict (e.g., CSS property list, JSON value type)

4. File Type Registry & Skip Lists (§2)

Two dictionaries map files to parsers before any parsing begins:

FILE_REGISTRY (extension-based)

Covers 50+ file extensions. Each entry is a 3-tuple:

```
extension → (file_type, language, parser_key)
```

Example: `.py → ("code", "python", "python")` , `.ts → ("code", "typescript", "treesitter")` , `.yaml → ("config", "yaml", "yaml")`

FILENAME_REGISTRY (exact filename-based)

Handles special files without extensions, such as Dockerfile, Makefile, .env, .gitignore, and requirements.txt.

Skip Lists

Three filters prevent wasted processing:

- SKIP_EXTENSIONS — binary/media types: images, audio, video, archives, compiled files, fonts, databases
- SKIP_DIRS — generated/vendor directories: node_modules, .git, __pycache__, dist, build, .venv, etc.
- SKIP_FILENAME_PATTERNS — minified files (*.min.js), lock files (package-lock.json, yarn.lock, Cargo.lock, etc.)

5. Core Helpers (§3)

_safe_read(path)

Reads any file safely with automatic encoding detection. It first sniffs the raw bytes for null characters (binary signal). If chardet is available it detects the encoding; otherwise defaults to UTF-8. Replaces undecodable bytes rather than crashing. Returns None for binary files.

_chunk_id(rel_path, kind, name, counter)

Generates a deterministic, human-readable chunk ID. Sanitizes non-word characters from both the filename stem and symbol name, then combines them:

```
{stem}__{kind}__{name}__{counter}
```

_line_block_chunks(source, rel_path, language, file_type, block_size=60)

The universal fallback for files without a dedicated parser. Splits source into consecutive 60-line blocks, producing one Chunk per block. Also used as the error fallback inside dedicated parsers when they encounter malformed input (e.g., a Python SyntaxError).

6. Dedicated Parsers (§4)

Each parser class exposes a single parse(source, rel_path) → List[Chunk] method. All parsers fall back to _line_block_chunks on error, ensuring the pipeline never hard-crashes on a malformed file.

6a. PythonParser — Python AST

Uses Python's built-in ast module for the most accurate Python parsing available.

- Parses the source into an AST with ast.parse().
- Annotates every AST node with a _parent pointer via ast.walk().
- Extracts ClassDef nodes → chunk_type: class
- Extracts FunctionDef / AsyncFunctionDef nodes → chunk_type: function or method (if inside a class)
- For each function: extracts parameters (excluding self/cls), return type annotation (ast.unparse), and docstring (ast.get_docstring).
- Extracts all import statements (import X and from X import Y) into the imports field.
- Fallback: SyntaxError → _line_block_chunks.

6b. TreeSitterParser — JS/TS/Java/C/C++/Go/Rust

Uses the tree-sitter library with official language grammars for compiled/web languages.

- Gracefully degrades to line-block chunking if tree-sitter is not installed.
- Walks the parse tree recursively, matching FUNCTION_NODES and CLASS_NODES sets.
- Recognized function node types: function_declaration, arrow_function, method_definition, fn_item (Rust), func_literal (Go), and more.
- Recognized class node types: class_declaration, struct_item (Rust/C), impl_item (Rust), interface_declaration.
- Extracts parameter text directly from the syntax tree's parameters field.
- Caches one TreeSitterParser instance per language inside IngestionPipeline to avoid repeated initialization.

6c. HTMLParser — HTML / Vue / Svelte

Uses BeautifulSoup to decompose an HTML file into typed sub-chunks.

- Script blocks (<script>) → chunk_type: script_block, language: javascript
- Style blocks (<style>) → chunk_type: style_block, language: css
- Semantic structural tags (<section>, <article>, <main>, <header>, <footer>, <nav>, <aside>, <form>, <table>, <dialog>, <template>) → chunk_type: html_section
- Remaining body text → chunk_type: html_text, file_type: doc
- Approximate line numbers computed by searching for the first 30 characters of the snippet in the original source.
- Fallback: BeautifulSoup not installed → _line_block_chunks.

6d. CSSParser — CSS / SCSS / LESS

Uses a regex-based rule-block extractor (no external library required).

- First pass: extracts all CSS custom properties (--variable: value) into a single css_variables chunk.
- Second pass: matches every rule block (selector { properties }) using a DOTALL regex that handles nested braces.
- Classifies each rule by selector prefix: @keyframes, @media, @mixin, @<other at-rule>, pseudo-class rules, and plain css_rule.
- The extra field contains the list of extracted property declarations and a property count.

6e. JSONParser

Parses JSON with the stdlib json module, then chunks by structure:

- Array → batches of 20 items each → chunk_type: json_array_batch
- Object → one chunk per top-level key → chunk_type: json_key
- Scalar → single chunk → chunk_type: json_value

6f. YAMLParser

Requires pyyaml (optional). Parses YAML and creates one chunk per top-level key/section. Falls back gracefully if pyyaml is not installed.

6g. TOMLParser

Supports Python 3.11+ stdlib tomllib or the tomli backport. Produces one chunk per top-level TOML section/key.

6h. INIParser

Handles .ini, .cfg, .conf, and .env files using Python's stdlib configparser. One chunk per [section]; an additional chunk for bare key=value pairs with no section header (DEFAULT).

6i. XMLParser

Uses stdlib xml.etree.ElementTree. Iterates all child elements of the root, producing one chunk per direct child element. The tag name and attributes are included in extra.

6j. CSVParser

Reads CSV/TSV files using Python's csv.DictReader. Groups rows into batches of 50, with the full header row retained in every chunk for context. The extra field records total row count and batch position.

6k. NotebookParser — Jupyter .ipynb

Parses Jupyter notebook JSON directly. Extracts each cell independently with its cell_type (code or markdown). For code cells, the execution_count is stored in extra.

6l. ShellParser — Bash/Zsh/Fish

Uses a regex-based approach to extract shell functions. Recognizes both function name() { and name() { syntax. Also collects leading comment blocks as the chunk's docstring-equivalent. Falls back to line-block chunking if no functions are found.

6m. SQLParser

Splits a SQL file by statement-level keywords: CREATE TABLE, CREATE VIEW, CREATE FUNCTION, CREATE PROCEDURE, CREATE INDEX, SELECT, INSERT, UPDATE, DELETE, ALTER, DROP, WITH, CALL. Each matched statement becomes a separate chunk with its SQL keyword as the chunk_type.

6n. MarkdownParser

Splits Markdown files at heading boundaries (# H1, ## H2, etc.). Each section (from one heading to the next) becomes one chunk. The heading text becomes the chunk name and the heading level is stored in extra.

6o. RSTParser — reStructuredText

Splits .rst files at underline-based section markers (e.g., lines of === or ---). Each resulting section becomes one chunk.

6p. PlainTextParser

Splits plain .txt files at double-newline paragraph boundaries. Each paragraph becomes one chunk with chunk_type: paragraph.

6q. PDFParser — PyMuPDF

Requires pymupdf (pip install pymupdf). Processes PDF files differently from all other parsers — it receives the file path rather than text content. Iterates each page, extracting text blocks via page.get_text("blocks"). Each block becomes a chunk with the page number and block index stored in extra.

6r. DockerfileParser

Splits a Dockerfile at instruction keywords (FROM, RUN, COPY, ADD, CMD, ENTRYPOINT, ENV, ARG, EXPOSE, WORKDIR, LABEL, USER, VOLUME, ONBUILD, STOPSIGNAL, HEALTHCHECK). Each instruction group becomes a chunk.

6s. MakefileParser

Splits a Makefile at target boundaries. A target is detected by a non-whitespace line followed by a colon (e.g., build:). Everything from one target line to the next becomes one chunk.

7. Repo Loader (§5)

RepoLoader handles two input modes:

Local path mode

Resolves and validates the local_path, returning it as a Path object for downstream processing.

GitHub clone mode

Runs `git clone --depth 1 <url>` into a temporary directory. The shallow clone (depth 1) minimizes download time and storage. Raises RuntimeError if git returns a non-zero exit code.

collect_files(root)

Recursively walks the repo root with rglob("*"). For each file:

- Skips directories in SKIP_DIRS and filenames matching SKIP_FILENAME_PATTERNS.
- Skips extensions in SKIP_EXTENSIONS.
- Looks up the file in FILE_REGISTRY (by extension) then FILENAME_REGISTRY (by lowercased filename).
- Files not found in either registry are tagged as generic_text and handled by the line-block fallback.
- Returns a list of (abs_path, file_type, language, parser_key) tuples.

8. IngestionPipeline — Orchestrator (§6)

IngestionPipeline is the top-level class that wires all components together.

Initialization

Instantiates one of every parser class and stores them in a _parsers dict keyed by parser_key. Tree-sitter parsers are lazy-cached in _ts_cache (one instance per language, created on first use).

run()

The main execution loop:

- Calls loader.load() to get the repo root path.
- Calls loader.collect_files() to get the list of files to process.
- For each file: checks size against max_bytes (default 500 KB) and skips oversized files.
- Calls _dispatch() to route the file to the correct parser.
- Accumulates all chunks and tracks per-language counts and error statistics.
- Logs a final summary table: total chunks, skipped files (by reason), and chunks per language.

_dispatch(abs_path, rel_path, file_type, language, parser_key)

Routes each file to the appropriate parser:

- PDF files: passed directly to PDFParser (which needs the file path, not decoded text).
- All other files: decoded with _safe_read(). Returns None (binary), [] (empty), or the parsed chunks.
- Known parser_key → looked up in _parsers dict.
- parser_key == "treesitter" → routed to the cached TreeSitterParser for that language.
- Unknown parser_key → _line_block_chunks fallback.

9. Serialisation (§7)

Two functions write the final chunk list to disk:

Function	Output Format
chunks_to_json(chunks, path)	Single JSON array file with 2-space indentation. Best for debugging and small repos.
chunks_to_jsonl(chunks, path)	JSON Lines file (one JSON object per line). Preferred for large repos and streaming ingestion in downstream stages.

The CLI automatically selects JSONL if the --output filename ends in .jsonl, otherwise writes JSON.

10. CLI Usage (§8)

The script is fully runnable from the command line via argparse:

Argument	Description
--repo_url	GitHub URL to clone and ingest (mutually exclusive with --local_path)
--local_path	Path to a local repository directory
--output	Output file path (default: chunks.json). Use .jsonl extension for JSONL format.
--max_file_size_kb	Maximum file size in kilobytes to process (default: 500 KB). Larger files are skipped.

Example Commands

```
python ingestion.py --repo_url https://github.com/ArulKevin2004/VisionCortex
python ingestion.py --local_path /path/to/repo --output chunks.jsonl
python ingestion.py --local_path /path/to/repo --max_file_size_kb 1000
```

11. Dependencies & Graceful Degradation

All non-stdlib dependencies are optional. The module detects availability at import time and degrades gracefully:

Package	Impact if Missing

pyyaml	YAML files fall back to line-block chunking
chardet	Encoding detection defaults to UTF-8 (may misread some files)
pymupdf (fitz)	PDF files are skipped entirely
beautifulsoup4	HTML files fall back to line-block chunking
tomllib / tomli	TOML files fall back to line-block chunking (tomllib is stdlib in Python 3.11+)
tree-sitter + grammars	JS/TS/Java/C/C++/Go/Rust fall back to line-block chunking

Install all optional dependencies:
pip install pyyaml chardet pymupdf beautifulsoup4 tomli tree-sitter tree-sitter-javascript tree-sitter-java tree-sitter-c tree-sitter-cpp tree-sitter-go tree-sitter-rust

12. Sample Output Chunk

Every chunk written to the output file follows this JSON schema:

```
{
  "chunk_id":    "ingestion__function__parse__3",
  "content":     "def parse(self, source: str, ...) -> list[Chunk]: ...",
  "chunk_type":  "function",
  "file_type":   "code",
  "language":    "python",
  "source_file": "stage1/ingestion.py",
  "start_line":  521,
  "end_line":    580,
  "name":        "parse",
  "docstring":   "Parses HTML into typed sub-chunks.",
  "parameters":  ["source", "rel_path"],
  "return_type": "list[Chunk]",
  "parent":      "HTMLParser",
  "imports":     ["bs4", "re", "pathlib"],
  "extra":       {}
}
```

13. End-to-End Data Flow

The complete journey from repository to structured chunks:

- User invokes CLI with --repo_url or --local_path.
- IngestionPipeline creates a RepoLoader and one parser instance per supported format.
- RepoLoader either clones the repo via git (shallow) or validates the local path.
- collect_files() walks the file tree, applying all skip filters, and tags each file with its (file_type, language, parser_key).
- The main run() loop iterates every tagged file.
- Files exceeding max_file_size_kb are logged and skipped.
- _dispatch() reads the file text (with safe encoding detection) and routes it to the matching parser.

- Each parser produces a List[Chunk] using language-specific AST/syntax/structure analysis; malformed inputs fall back to line-block chunking.
- All chunks are accumulated, counted by language, and logged.
- The final list is serialised to JSON or JSONL.
- Stage 2 (NLP Pipeline) reads the output file to embed, index, and serve knowledge transfer.